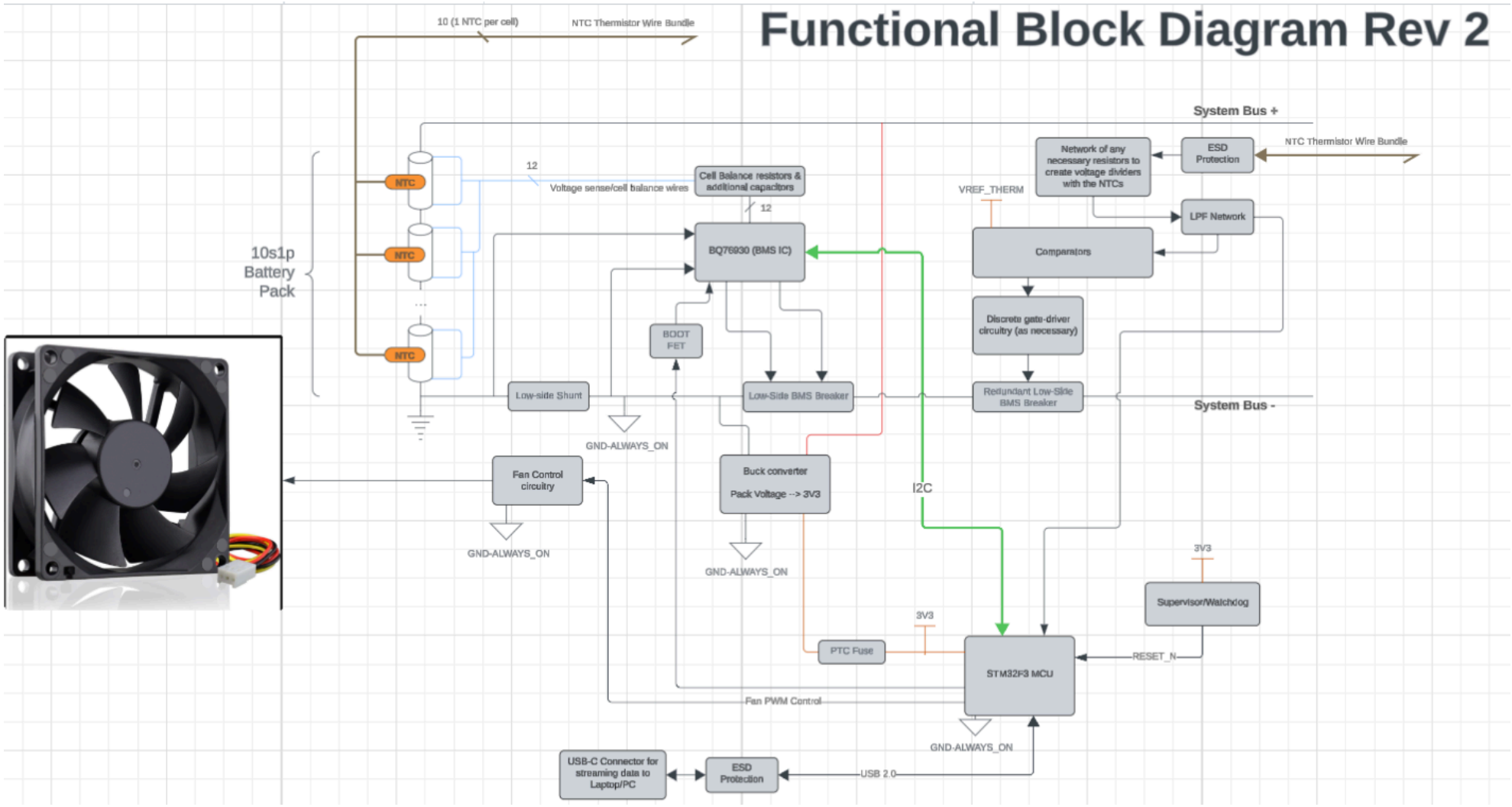
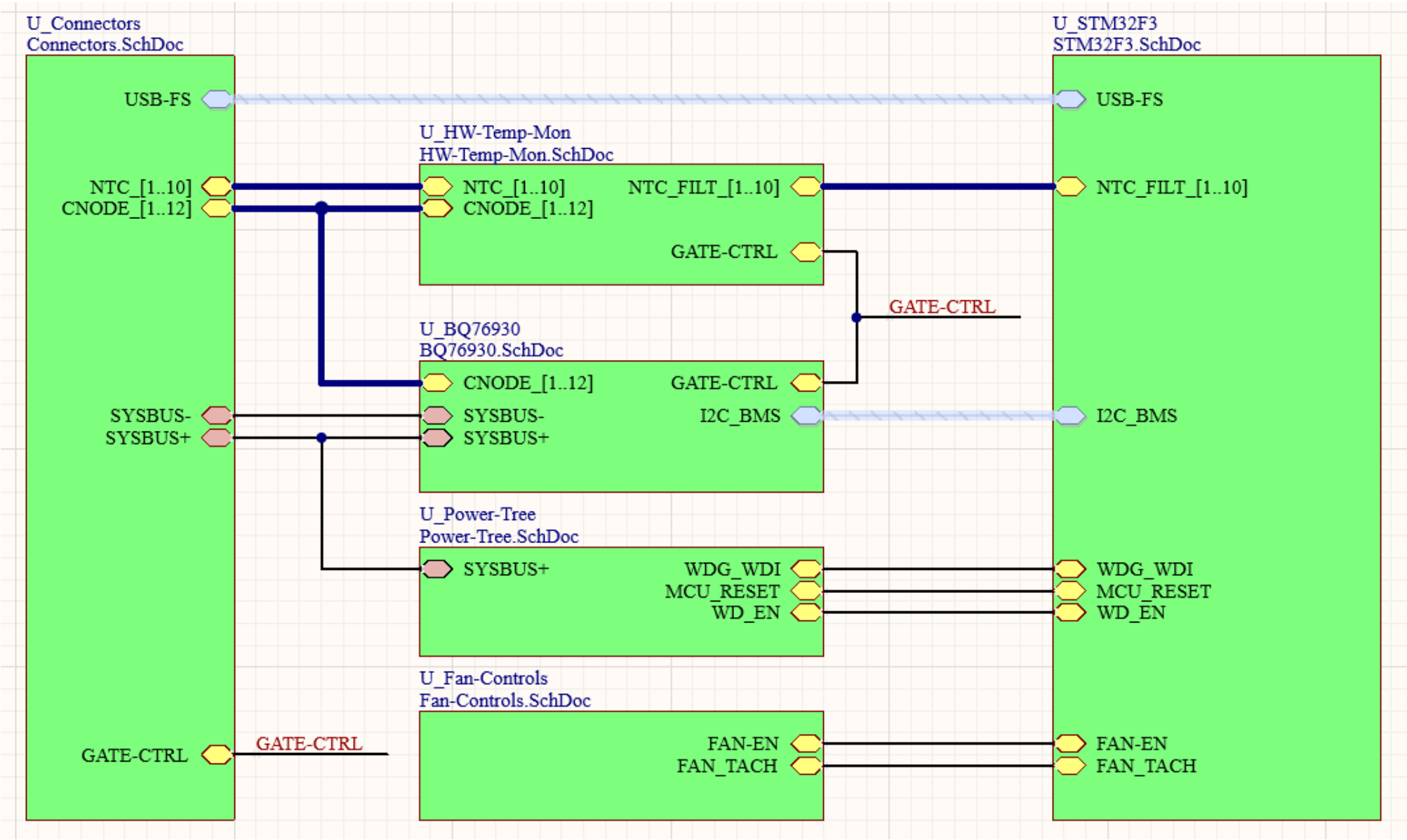
Senior Design Battery Management System Functional Overview

By Christian, Connor, Nick, and Kaushik

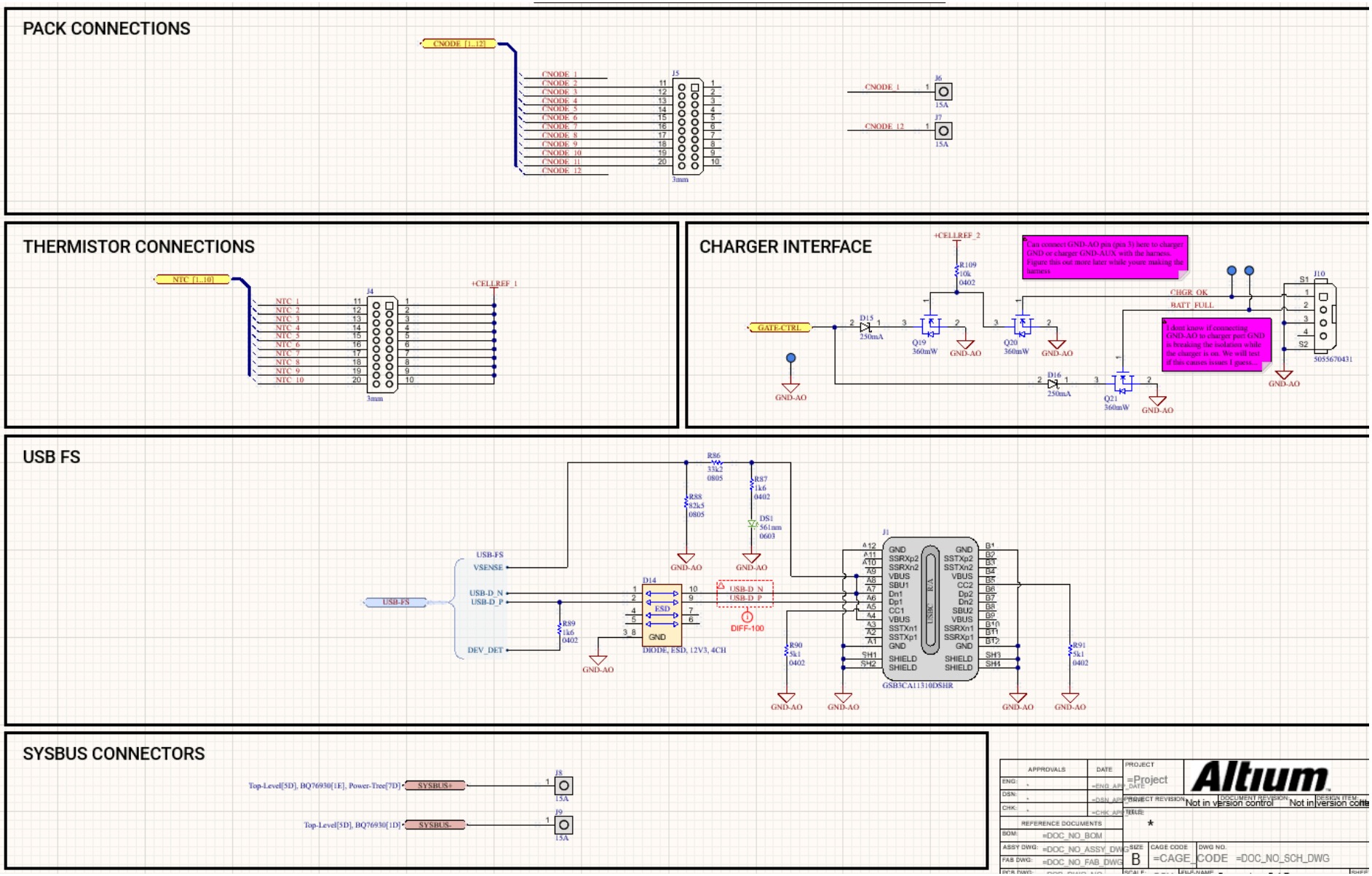
Functional Block Diagram (available on LucidChart here)



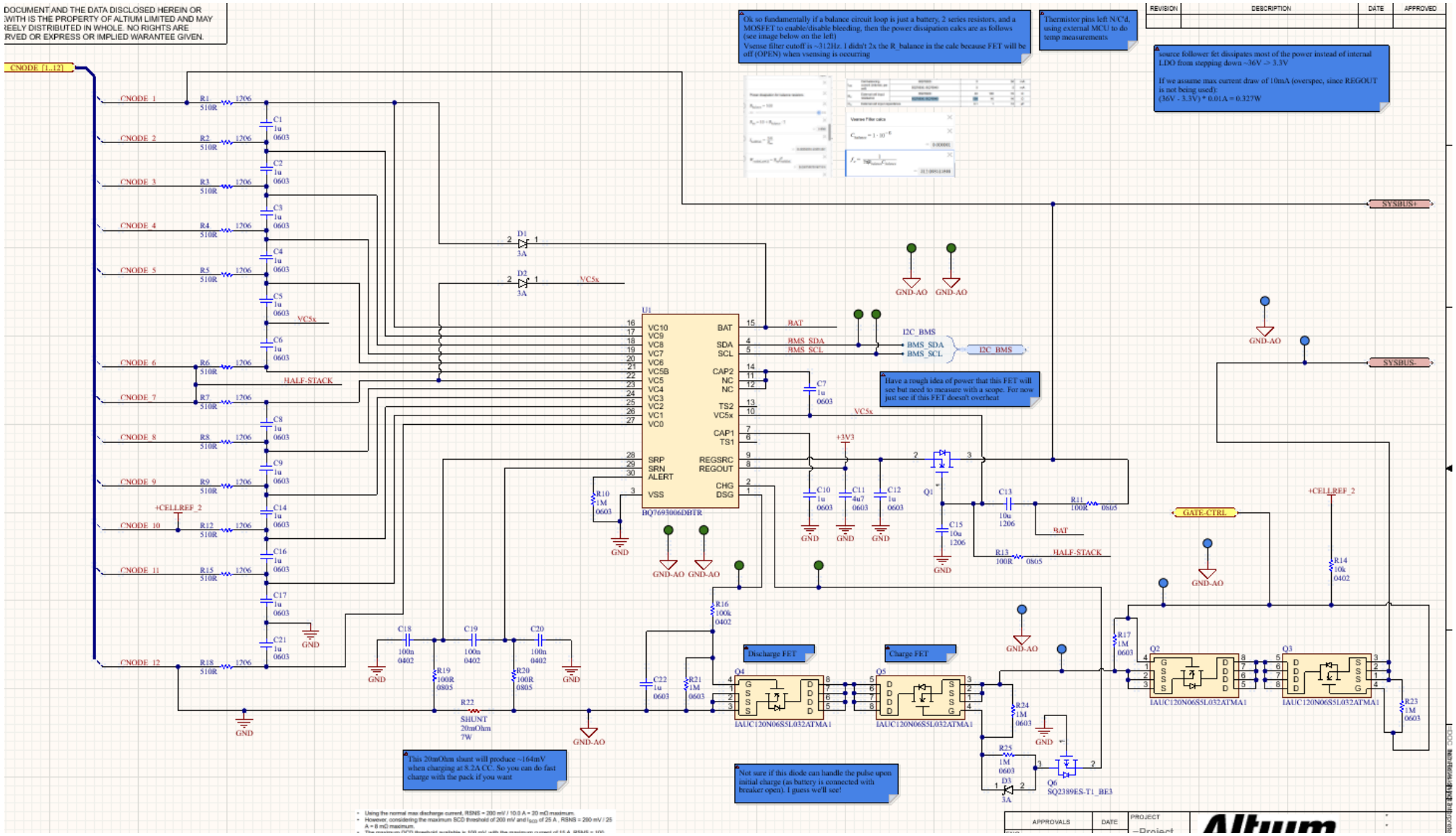
Top-Level.SchDoc



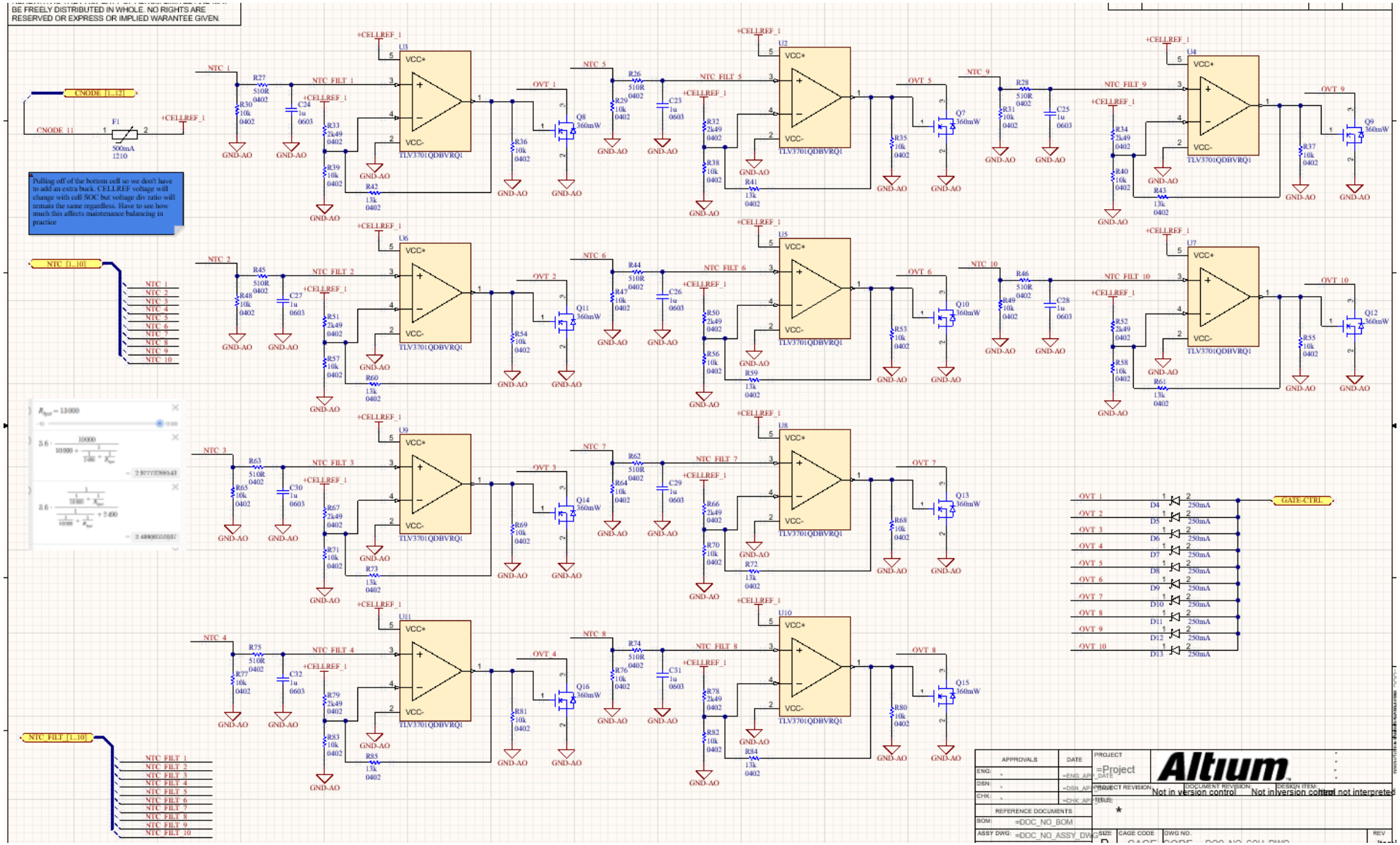
Connectors.SchDoc



BQ76930.SchDoc

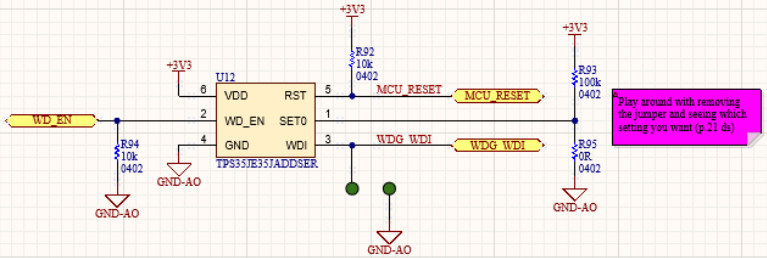


HW-Temp-Mon.SchDoc

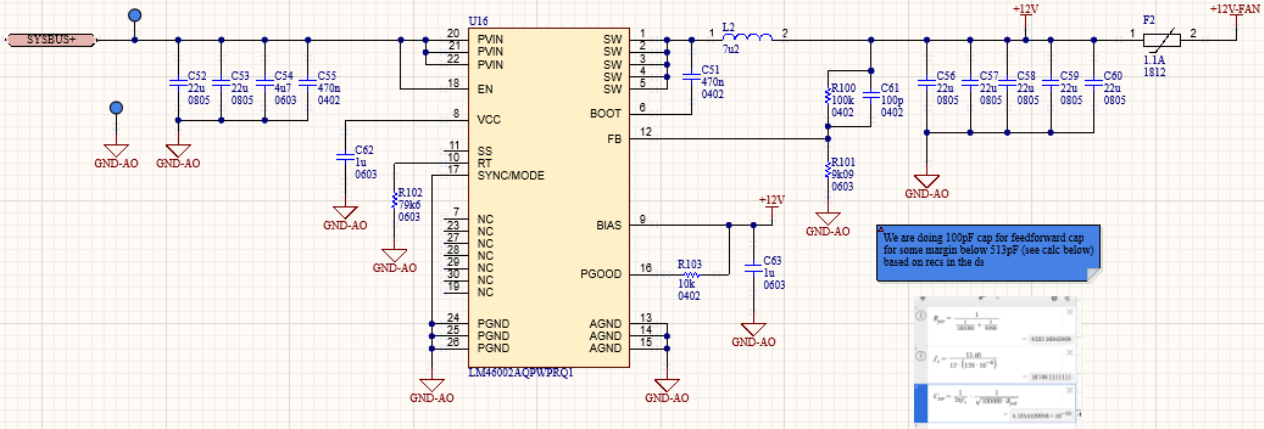


Power-Tree.SchDoc

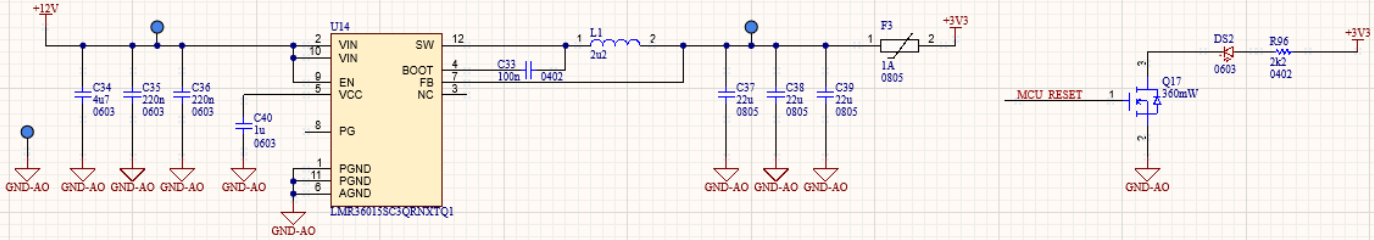
SUP/WATCHDOG



PACK --> 12V

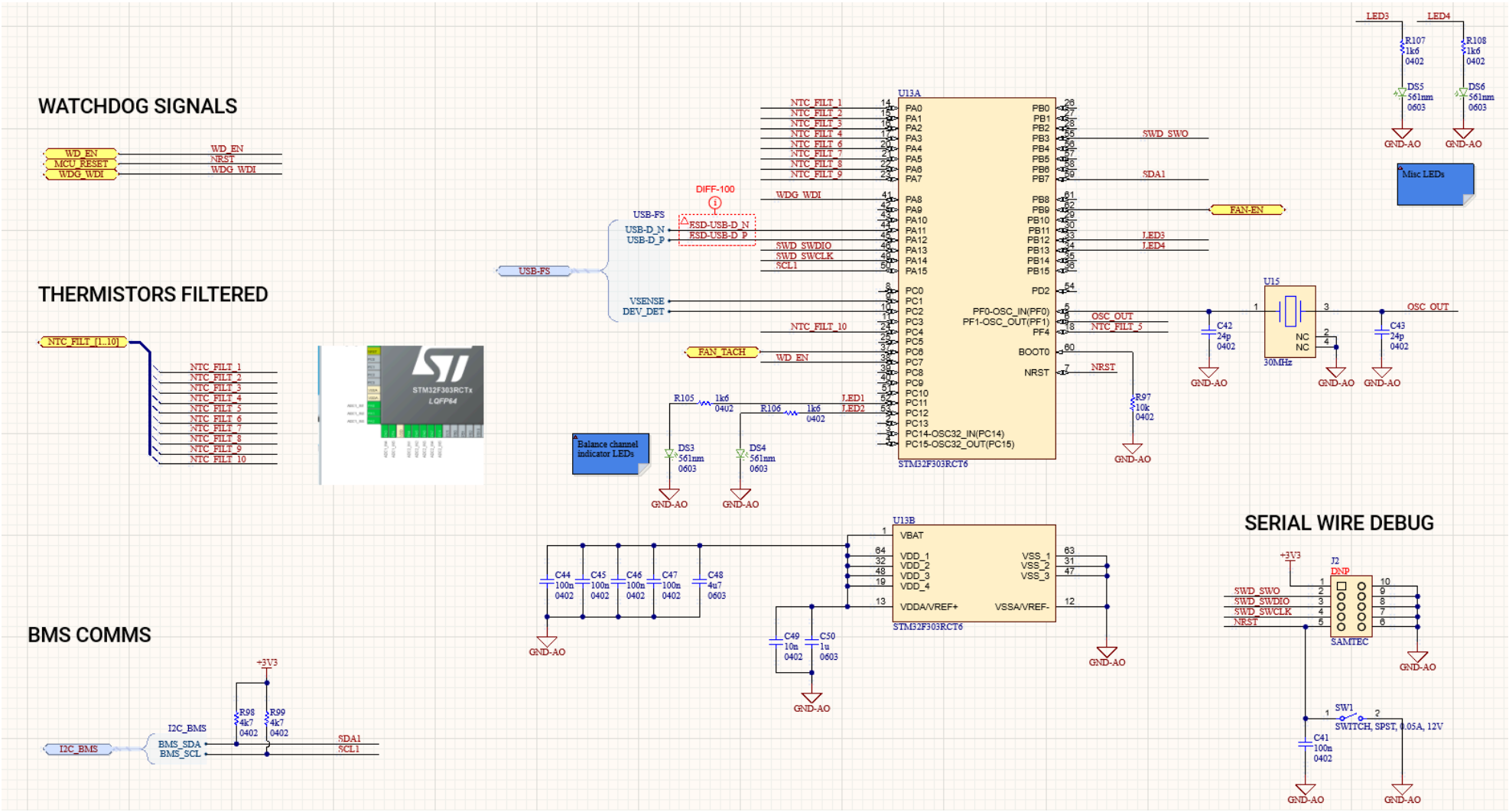


12V --> 3V3

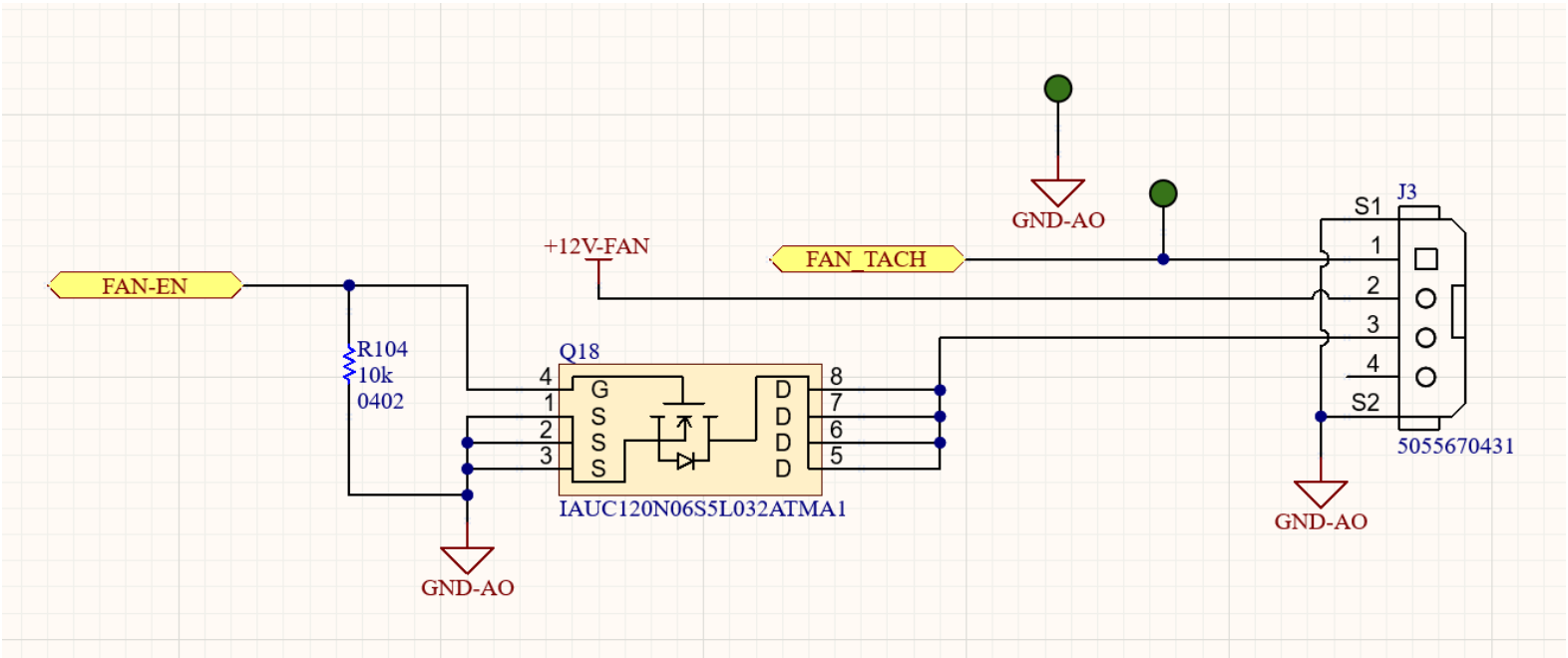


SCHEMATIC REVIEW CHECKLIST:
- Test points?

STM32F3.SchDoc



Fan-Controls.SchDoc



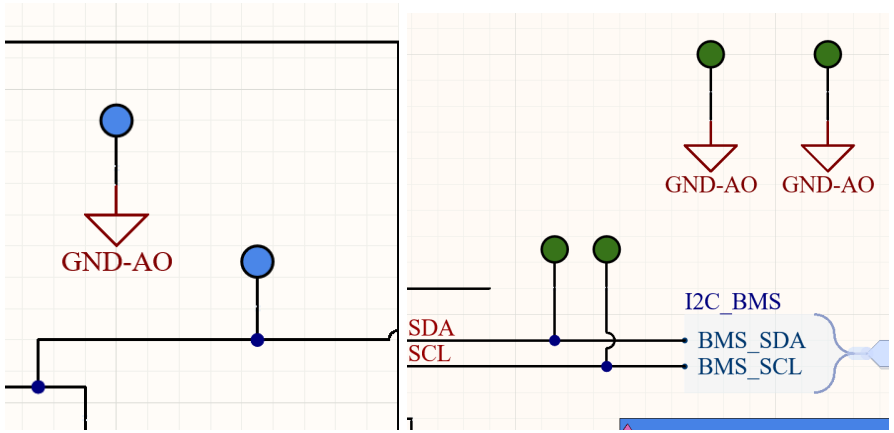
Functional Block Diagram Updates

- I decided to remove the galvanic isolation between the power/BMS section and the MCU/comms section because that may actually be unnecessary for this specific situation. The battery pack voltage is, at worst, going to be 42V, so we aren't dealing with that high of a voltage. Additionally, based on our mechanical packaging strategy, the MCU/USB/comms components are going to be on the other side of the PCB from the battery pack terminals and BMS stuff. Additionally, I have an old laptop that I don't care about that is running Linux that I would be comfortable connecting the BMS to for our testing and data collection. So either way, if catastrophic failure happens somehow with our BMS and kills the laptop or its USB port, it doesn't matter because it's an old laptop that I don't care about. Based on my calculations it was looking probable that we would be forced to design a flyback converter if we continued to use galvanic isolation in our circuit because all of the isolated DC-DC modules I found could only supply a small amount of current. We don't know how much current the STM32 microcontroller will draw when its running our finalized firmware, and we want to make sure we don't encounter any weird power-related issues that cause problems that are hard to diagnose. In my experience I have had weird stuff happen before like the STM32 getting stuck in bootloader due to power issues and giving weird behavior that took me an entire week to diagnose. So long as we practice safe handling with our battery pack and use an old laptop, it should be fine for our purposes if we remove the galvanic isolation. This will allow us to make a functional prototype faster, and if we want to improve the design afterwards by adding galvanic isolation then we can do so with minimal issues because the rest of the design will already be proven out.

Schematic Explanation

Notes

- There are various types of test points in the circuit (see image below). Green ones are just exposed copper circles, while blue ones are through holes with exposed copper. We can easily solder to the blue test points, green ones are a bit harder to solder to but still doable. I've done it before on my other projects.



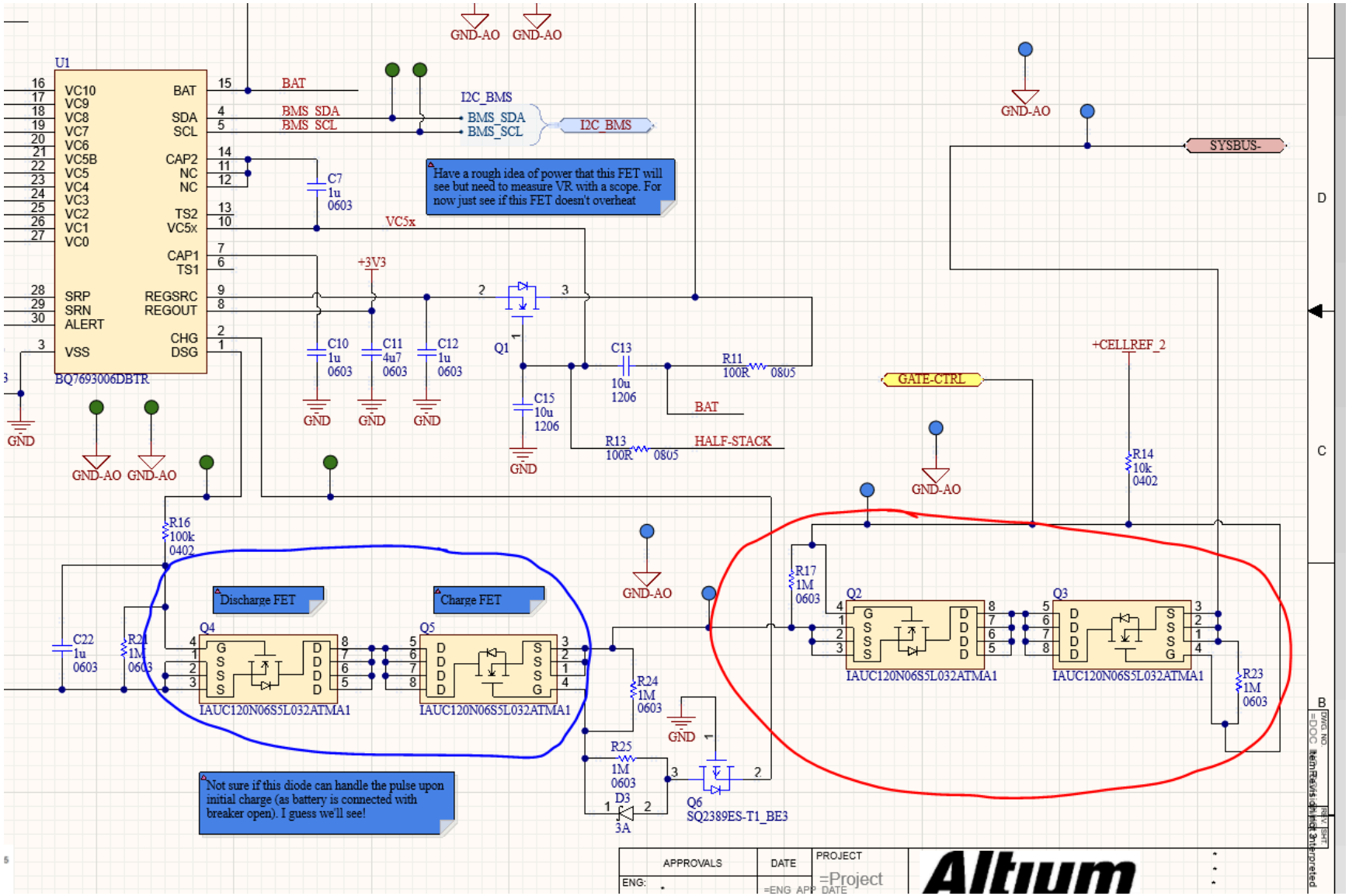
Top-Level.SchDoc

- Basically just using Altium's hierarchical design feature to tie together all of the lower level schematics in a way that also kinda looks like a block diagram

Connectors.SchDoc

- You can see that there is a connector for interfacing with the charger. The charger we are using is the NPB-450-24 by MEAN WELL. You can download the datasheet from the official website here. We are also purchasing the SBP-001, which is another MEAN WELL product that connects to the charger's CAN bus interface and converts it to USB so we can connect the charger to our laptop and use the MEAN WELL charger software to set charge profiles.
- The charger has a CHARGER OK signal and a BATTERY FULL signal. I wanted to use those signals to open the redundant BMS breaker FETs to provide a hardware stop for charging if the battery is full or if there is a charger fault. The BMS breakers can be found on the

BQ76930.SchDoc schematic page. The redundant one is circled in red (see below image) and main one is circled in blue.

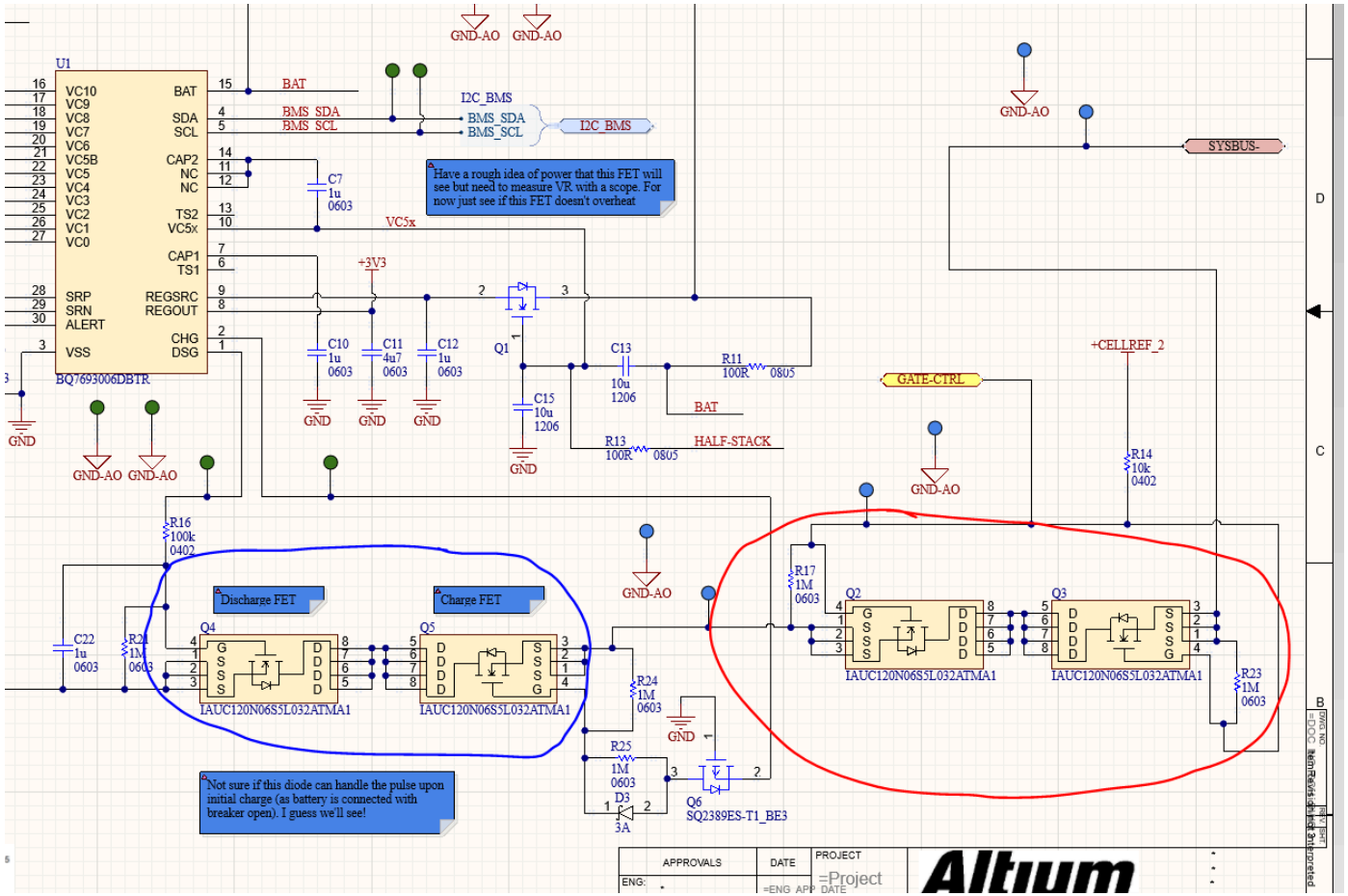


- The BMS has the USB connection for connecting to laptop. Relevant connectors have ESD protection (mainly USB).

BQ76930.SchDoc

- We tried to select the simplest BMS IC we could find for our project so we could make our project work as fast as possible. We found this one, and you interface with it via I2C. So it's very simple and it makes PCB routing easier because its a very low speed interface. It has a built in shunt amplifier and we are using a 20mOhm, 7W shunt as you can see in the schematics (it's the red colored resistor).
- We have two BMS breakers, which in this case are just for gating current flow through the battery pack system power bus (SYSBUS). If we want to charge the battery pack, we put the BMS in charging mode and it drives the main BMS breaker FETs (circled in blue in image below) such that current can only flow into the battery pack. If we put the BMS in discharging mode, then the FETs are driven such that current can only flow out of the pack. I did some simulations of this under the assumption that the pack is connected to a power bus with lots of capacitance, so I added some RC filters to the main BMS breaker FETs to control the slew rate of the FETs to act similarly to the soft-start function of a buck converter, so we can prevent inrush issues if turning on the pack when connected to a large capacitive load.

Not sure how necessary or useful that's actually going to be but I guess we'll see.



HW-Temp-Mon.SchDoc

- Basically it's just a bunch of push pull comparators where I added hysteresis for controlling the redundant BMS breaker FETs. The NTC thermistors are 10k range thermistors but based on how I set the voltage thresholds the thermistors will toggle at around 60 degrees celsius. My hysteresis calculations are included later in the report via screenshots.

- We are powering the comparators and setting the reference voltages from the bottom battery cell instead of pulling off of a separate regulator for that whole group of components. The whole circuit should not draw that much current. This will slightly affect how the other cells are balanced but it might be fine. Just have to test. I want to save all the current from the 3V3 buck for the microcontroller in case it needs it.

Power-Tree.SchDoc

- This page has a power rail supervisor/watchdog IC on it that I have used before for a previous project. I already know how it works and have proven that out, I got it working in my previous project and have characterized the IC. I copied and pasted this from the schematics I made from a previous project I made. My implementation for everything here was done from scratch, and anything I copied and pasted into these BMS schematics were copied and pasted from my own schematics in past projects that were also done from scratch.
- There is a big buck converter on here to power the fan at 12V. There is then a downstream 3V3 buck regulator which I reused from a previous project of mine that I like a lot. I added a PTC fuse for the fan itself and another for the 3V3 rail. This ensures any accidental shorting is easily observable

STM32F3.SchDoc

- I am reusing a microcontroller I used in a previous project with USB-C. This part of the design is also proven to work. We are using SWD instead of JTAG for no particular reason. It's a bit arbitrary in this case which one we use. We have a few extra indicator LEDs that we can use to indicate things with the STM32.

Fan-Controls.SchDoc

- This has the simple control circuitry for the two fans and a connector. We decided to use two fans after I finished the schematics. Luckily the 12V regulator can handle up to 2A which is more than enough to drive two fans. We dont have to add an extra connector, we can just modify the harness a bit so we don't have to change the schematics. It's a little bit janky to splice the harness in general but it doesn't matter for us.

BMS Calculations (slightly disorganized)

See the other PDF attached to the email I sent you